1 Datarensning

```r
pacman::p_load(tidyverse, lubridate, forcats, readr)

# Indlæs data
cancellation <- read.csv("data/cancellation.csv")
subscription <- read.csv("data/subscription_v2.csv", sep = ";")
# behavior <- read.csv("data/behavior.csv")

# Behold kun 1 ID per row (seneste dato pr. tabel)
# cancellation → brug expiration_date
cancellation <- cancellation %>%
  mutate(expiration_date = ymd(expiration_date)) %>%
  group_by(pseudo_id) %>%
  slice_max(expiration_date, n = 1, with_ties = FALSE) %>%
  ungroup()

# subscription → brug subscription_cancel_date
subscription <- subscription %>%
  mutate(subscription_cancel_date = dmy(subscription_cancel_date)) %>%
  group_by(pseudo_id) %>%
  slice_max(subscription_cancel_date, n = 1, with_ties = FALSE) %>%
  ungroup()

# Flet data + churn-definition
merged_data <- subscription %>%
  left_join(cancellation, by = "pseudo_id") %>%

  # Parse dates
  mutate(
    subscription_cancel_date = as.Date(subscription_cancel_date),
    first_campaign_day = dmy(first_campaign_day),
    last_campaign_day = dmy(last_campaign_day),
    expiration_date = as.Date(expiration_date)
  ) %>%

  # Churn
  mutate(
    churn = case_when(
      is.na(expiration_date) ~ 0,
      expiration_date <= last_campaign_day ~ 1,
      expiration_date > last_campaign_day ~ 0
    )
  ) %>%

  # Continued subscription
  mutate(
    continued_subscription = 1 - churn
  ) %>%

  # Early churn
  mutate(
    early_churn = case_when(
      continued_subscription == 1 & !is.na(expiration_date) &
```

```
        expiration_date <= last_campaign_day + 90 ~ 1,
      continued_subscription == 1 ~ 0,
      continued_subscription == 0 ~ 0
    )
  )

# Churn distribution
merged_data %>%
  count(churn) %>%
  mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
  churn     n  prop
  <dbl> <int> <dbl>
1     0   742 0.580
2     1   537 0.420
```

```
# Feature engineering / rensning
model_data <- merged_data %>%
  mutate(
    order_date = as.Date(ymd_hms(order_date)),
    birthdate = as.Date(parse_date_time(birthdate, orders = c("dmy", "ymd", "mdy"))),
    usr_created = dmy(usr_created)
  ) %>%

  # Fjern NA
  filter(!is.na(birthdate), !is.na(usr_created), !is.na(order_date)) %>%

  # Kategorisk rensning
  mutate(
    churn = factor(churn),
    type = factor(type),
    reason = factor(reason),
    koen = factor(koen),
    koen = fct_recode(koen, "Ikke oplyst" = ""),
    type = fct_na_value_to_level(type, level = "Ingen afmelding"),
    reason = fct_na_value_to_level(reason, level = "Ingen afmelding"),
    expiration_date = replace_na(expiration_date, as.Date("3000-01-01"))
  ) %>%

  # Samtykker
  mutate(
    permission_given_order = factor(if_else(permission_given_order == "true", 1, 0)),
    permission_given_today = factor(if_else(permission_given_today == "true", 1, 0))
  ) %>%

  # Afledte variable
  mutate(
    age = as.numeric(Sys.Date() - birthdate) / 365,
    kundetid_dage = as.numeric(order_date - usr_created)
  ) %>%

  # Ekstra sikkerhed
  filter(!is.na(kundetid_dage)) %>%
```

```
  # Kundetid grupper
  mutate(
    kundetid_gruppe = case_when(
      kundetid_dage <= 7 ~ "0-7 dage",
      kundetid_dage <= 30 ~ "8-30 dage",
      kundetid_dage <= 180 ~ "1-6 måneder",
      TRUE ~ "6+ måneder"
    ) %>% factor()
  )

# Fjern ubrugte kolonner
model_data <- model_data %>%
  select(
    -kundetid_dage,
    -order_date,
    -birthdate,
    -usr_created
  )

# Tjek
glimpse(model_data)
```

```
Rows: 1,275
Columns: 22
$ pseudo_id                <chr> "003a92354fb88e0e3e411bc290291b6a", "004b2a16~
$ account_active_days      <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ subscription_cancel_date <date> 2025-05-11, 2025-04-29, 2025-03-18, 2025-05-~
$ koen                     <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ order_trackertag         <chr> "utm_campaign=Marts25&utm_content=b&utm_mediu~
$ permission_given_order   <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today   <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, ~
$ previous_subscriptions   <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns       <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials          <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ first_campaign_day       <date> 2025-03-05, 2025-03-05, 2025-03-18, 2025-03-~
$ last_campaign_day        <date> 2025-05-04, 2025-05-04, 2025-05-17, 2025-05-~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order  <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ type                     <fct> Andet, Andet, Andet, Produkt, Tid, Andet, Tid~
$ reason                   <fct> kunden ønsker ikke at oplyse, Ingen årsag - i~
$ expiration_date          <date> 2025-06-04, 2025-05-04, 2025-05-17, 2025-06-~
$ churn                    <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ continued_subscription   <dbl> 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, ~
$ early_churn              <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age                      <dbl> 76.33699, 69.78630, 39.75342, 45.69041, 74.29~
$ kundetid_gruppe          <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~
```

```
summary(model_data$age)
```

```
    Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
  0.3041  50.9699  61.4959  60.2016  71.5397 143.0712
```

```
colSums(is.na(model_data))
```

```
                pseudo_id      account_active_days subscription_cancel_date
                        0                        0                        0
                    koen         order_trackertag   permission_given_order
                        0                        0                        0
  permission_given_today   previous_subscriptions       previous_campaigns
                        0                        0                        0
         previous_trials       first_campaign_day        last_campaign_day
                        0                        0                        0
newsletters_before_order  newsletters_after_order                     type
                        0                        0                        0
                  reason          expiration_date                    churn
                        0                        0                        0
  continued_subscription             early_churn                      age
                        0                        0                        0
         kundetid_gruppe
                        0
```

```
# Churn distribution
model_data %>%
  count(churn) %>%
  mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
  churn     n  prop
  <fct> <int> <dbl>
1 0       740 0.580
2 1       535 0.420
```

```
model_data %>%
  count(continued_subscription) %>%
  mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
  continued_subscription     n  prop
                   <dbl> <int> <dbl>
1                      0   535 0.420
2                      1   740 0.580
```

```
model_data %>%
  count(early_churn) %>%
  mutate(prop = n / sum(n))
```

```
# A tibble: 2 x 3
  early_churn     n  prop
        <dbl> <int> <dbl>
1           0  1037 0.813
2           1   238 0.187
```

```r
glimpse(model_data)
```

```
Rows: 1,275
Columns: 22
$ pseudo_id                <chr> "003a92354fb88e0e3e411bc290291b6a", "004b2a16~
$ account_active_days      <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ subscription_cancel_date <date> 2025-05-11, 2025-04-29, 2025-03-18, 2025-05-~
$ koen                     <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ order_trackertag         <chr> "utm_campaign=Marts25&utm_content=b&utm_mediu~
$ permission_given_order   <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today   <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, ~
$ previous_subscriptions   <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns       <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials          <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ first_campaign_day       <date> 2025-03-05, 2025-03-05, 2025-03-18, 2025-03-~
$ last_campaign_day        <date> 2025-05-04, 2025-05-04, 2025-05-17, 2025-05-~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order  <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ type                     <fct> Andet, Andet, Andet, Produkt, Tid, Andet, Tid~
$ reason                   <fct> kunden ønsker ikke at oplyse, Ingen årsag - i~
$ expiration_date          <date> 2025-06-04, 2025-05-04, 2025-05-17, 2025-06-~
$ churn                    <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ continued_subscription   <dbl> 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 0, 0, 1, ~
$ early_churn              <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age                      <dbl> 76.33699, 69.78630, 39.75342, 45.69041, 74.29~
$ kundetid_gruppe          <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~
```

```r
view(model_data)

# Eksport
write_csv(model_data, "data/model_data.csv")
saveRDS(model_data, "data/model_data.rds")
```

2 Klyngeanalyse

```r
# ----------------------------
# 1. Load packages
# ----------------------------
pacman::p_load(tidyverse, DataExplorer, ggpubr)

# ----------------------------
# 2. Load data
# ----------------------------
behavior <- read.csv("data/behavior.csv")
model_data <- readRDS("data/model_data.rds")

glimpse(behavior)
```

```
Rows: 265,456
Columns: 10
$ pseudo_id        <chr> "fcfd45344b1647471f5ec6194c220c3e", "743a9d6d44dfa092~
```

```
$ dt               <chr> "2025-03-13", "2025-03-13", "2025-03-13", "2025-04-17~
$ collector_tstamp <chr> "2025-03-13 03:05:31.679000", "2025-03-13 01:24:24.26~
$ dvce_type        <chr> "Mobile", "Computer", "Computer", "Mobile", "Computer~
$ os_family        <chr> "Android", "Mac OS X", "Mac OS X", "iOS", "Mac OS X",~
$ page_url_clean   <chr> "https://jyllands-posten.dk/", "https://jyllands-post~
$ refr_medium      <chr> "search", "", "internal", "internal", "", "", "intern~
$ page_restricted  <chr> "no", "no", "no", "no", "no", "no", "yes", "yes", "no~
$ scroll_depth     <dbl> 0.00, 0.00, 1.00, 1.00, 0.00, 0.00, 0.00, 1.00, 0.00,~
$ webpage_id       <chr> "d12e9d666f4f9a3a62573d8d82c8a7d3", "5bb6b8db23b4ab77~
```

```r
# ----------------------------
# 3. Create behavioral customer-level features
# ----------------------------
customer_behavior <- behavior %>%
  group_by(pseudo_id) %>%
  summarise(
    n_visits = n(),
    avg_scroll = mean(scroll_depth, na.rm = TRUE),
    share_mobile = mean(dvce_type == "Mobile"),
    share_desktop = mean(dvce_type == "Computer"),
    share_restricted = mean(page_restricted == "yes"),
    share_search = mean(refr_medium == "search"),
    share_internal = mean(refr_medium == "internal"),
    n_unique_pages = n_distinct(webpage_id)
  )
# ----------------------------
# 4. Prepare subscription features
# ----------------------------
subscription_features <- model_data %>%
  select(
    pseudo_id,
    account_active_days,
    previous_subscriptions,
    previous_campaigns,
    previous_trials,
    newsletters_before_order,
    newsletters_after_order,
    churn
  )
# ----------------------------
# 5. MERGE DATA
# ----------------------------
customer_df <- subscription_features %>%
  left_join(customer_behavior, by = "pseudo_id")

# Indsæt 0 i stedet for NA for de 70 "Spøgelses-brugere"
customer_df <- customer_df %>%
  mutate(across(
    c(n_visits, n_unique_pages, avg_scroll, starts_with("share_")),
    ~replace_na(.x, 0)
  ))

# sanity check - should now be 1,275
nrow(customer_df)
```

```
[1] 1275
```

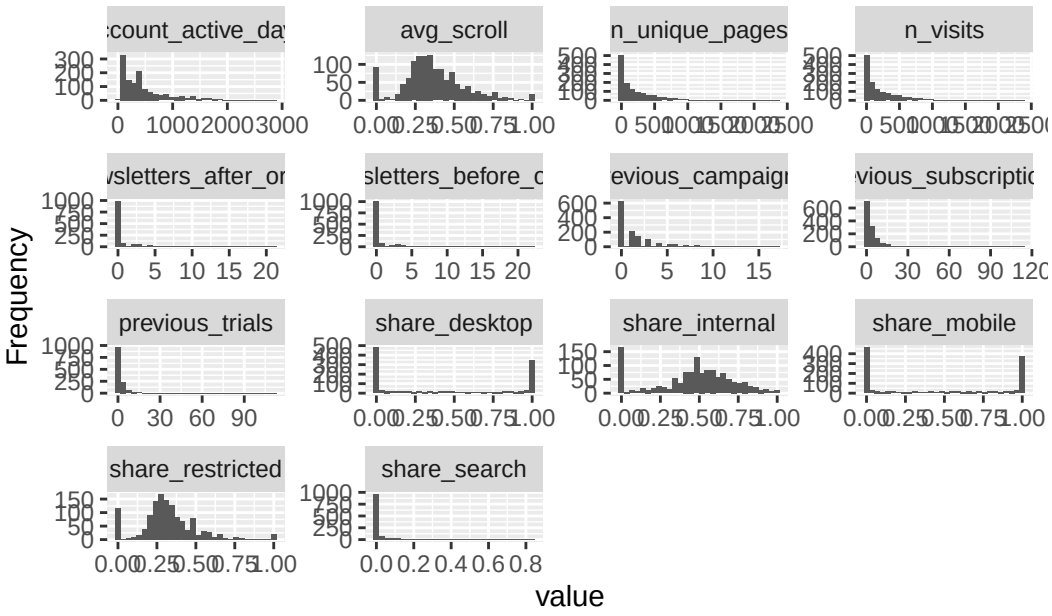
```r
# check
colSums(is.na(customer_df))
```

```
               pseudo_id      account_active_days   previous_subscriptions
                       0                        0                        0
      previous_campaigns          previous_trials newsletters_before_order
                       0                        0                        0
 newsletters_after_order                    churn                n_visits
                       0                        0                        0
              avg_scroll             share_mobile            share_desktop
                       0                        0                        0
        share_restricted             share_search           share_internal
                       0                        0                        0
          n_unique_pages
                       0
```

```r
# ----------------------------
# 6. Remove ID + handle missing values
# ----------------------------
cluster_input <- customer_df %>%
  select(-pseudo_id, -churn)

# check
glimpse(cluster_input)
```

```
Rows: 1,275
Columns: 14
$ account_active_days      <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ previous_subscriptions   <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns       <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials          <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order  <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ n_visits                 <int> 517, 603, 15, 8, 46, 253, 201, 15, 80, 772, 2~
$ avg_scroll               <dbl> 0.3471954, 0.4179104, 0.3666667, 0.6250000, 0~
$ share_mobile             <dbl> 0.47969052, 0.83084577, 1.00000000, 1.0000000~
$ share_desktop            <dbl> 0.5203095, 0.0000000, 0.0000000, 0.0000000, 1~
$ share_restricted         <dbl> 0.2978723, 0.3200663, 0.3333333, 0.6250000, 0~
$ share_search             <dbl> 0.015473888, 0.439469320, 0.000000000, 0.0000~
$ share_internal           <dbl> 0.52417795, 0.53565506, 0.46666667, 0.0000000~
$ n_unique_pages           <int> 516, 603, 15, 8, 46, 253, 201, 15, 80, 771, 2~
```

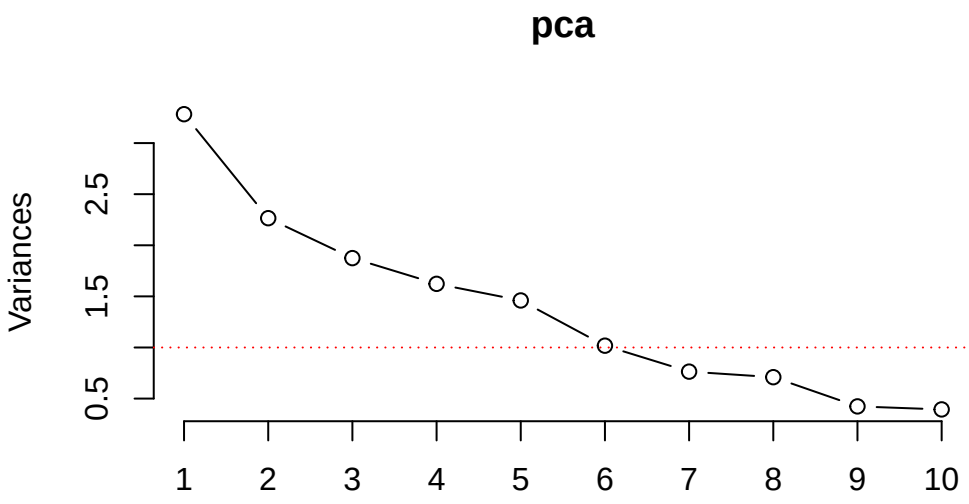
```r
# ----------------------------
# 7. Explore distributions
# ----------------------------
plot_histogram(cluster_input)
```



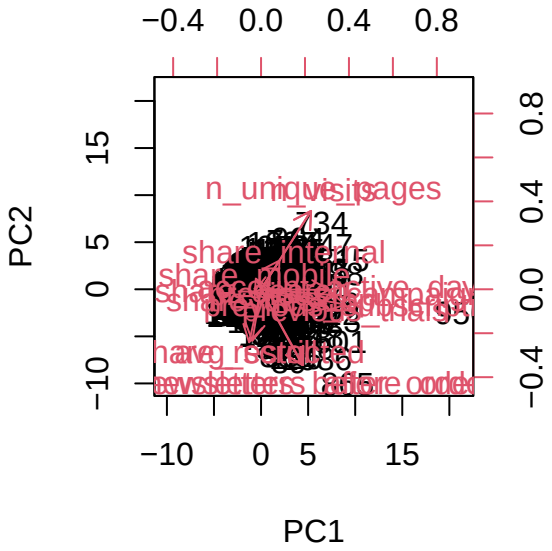

```
# ----------------------------
# 8. Scale variables
# ----------------------------
customer_scaled <- scale(cluster_input)

# ----------------------------
# 9. PCA
# ----------------------------
pca <- prcomp(customer_scaled, scale = TRUE)

summary(pca)
```

```
Importance of components:
                          PC1    PC2    PC3    PC4    PC5     PC6     PC7
Standard deviation     1.8118 1.5050 1.3691 1.2739 1.2084 1.00873 0.87394
Proportion of Variance 0.2345 0.1618 0.1339 0.1159 0.1043 0.07268 0.05455
Cumulative Proportion  0.2345 0.3963 0.5302 0.6461 0.7504 0.82304 0.87760
                           PC8     PC9    PC10    PC11    PC12    PC13    PC14
Standard deviation     0.84229 0.65074 0.62764 0.34835 0.22119 0.12194 0.04013
Proportion of Variance 0.05068 0.03025 0.02814 0.00867 0.00349 0.00106 0.00012
Cumulative Proportion  0.92827 0.95852 0.98666 0.99533 0.99882 0.99988 1.00000
```

```
screeplot(pca, type = "line")
abline(h = 1, col = "red", lty = 3)
```

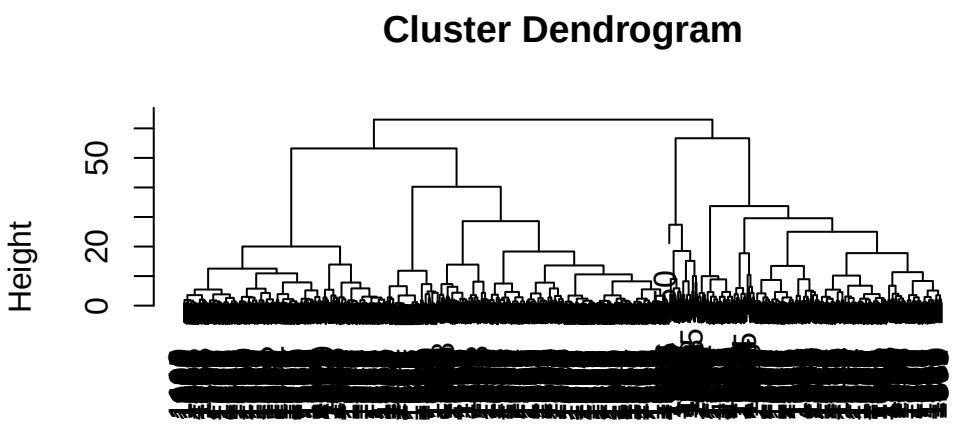



```
biplot(pca, scale = 0)
```




```
# ----------------------------
# 10. Hierarchical clustering
# ----------------------------
# hc_complete <- hclust(dist(pca$x[,1:4]), method = "complete")
# plot(hc_complete)
#
# customer_df <- customer_df %>%
#   filter(complete.cases(select(., -pseudo_id, -churn)))
#
```

```
# customer_df$cluster_hc <- cutree(hc_complete, 5)
#
# table(customer_df$cluster_hc)

hc_ward <- hclust(dist(pca$x[,1:4]), method = "ward.D2")
plot(hc_ward)
```




```
customer_df <- customer_df %>%
  filter(complete.cases(select(., -pseudo_id, -churn)))

customer_df$cluster_hc <- cutree(hc_ward, 5)

table(customer_df$cluster_hc)
```

```
  1   2   3   4   5
411 387 341  48  88
```

```
# ----------------------------
# 11. K-means clustering
# ----------------------------
set.seed(1)

km <- kmeans(pca$x[,1:4], centers = 5, nstart = 20)

customer_df$cluster_km <- km$cluster

table(customer_df$cluster_km)
```

```
  1   2   3   4   5
249 430 133 363 100
```

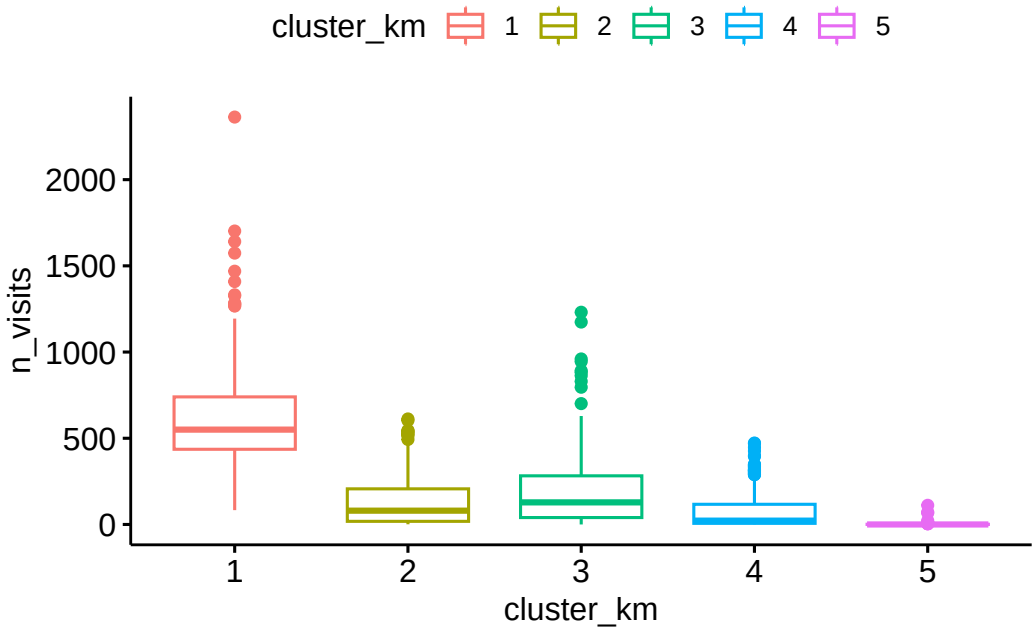
```
# compare methods
table(customer_df$cluster_km, customer_df$cluster_hc)
```

```
     1   2   3   4   5
  1 245   4   0   0   0
  2  81  18 331   0   0
  3  48  34   4  47   0
  4  36 321   5   1   0
  5   1  10   1   0  88
```

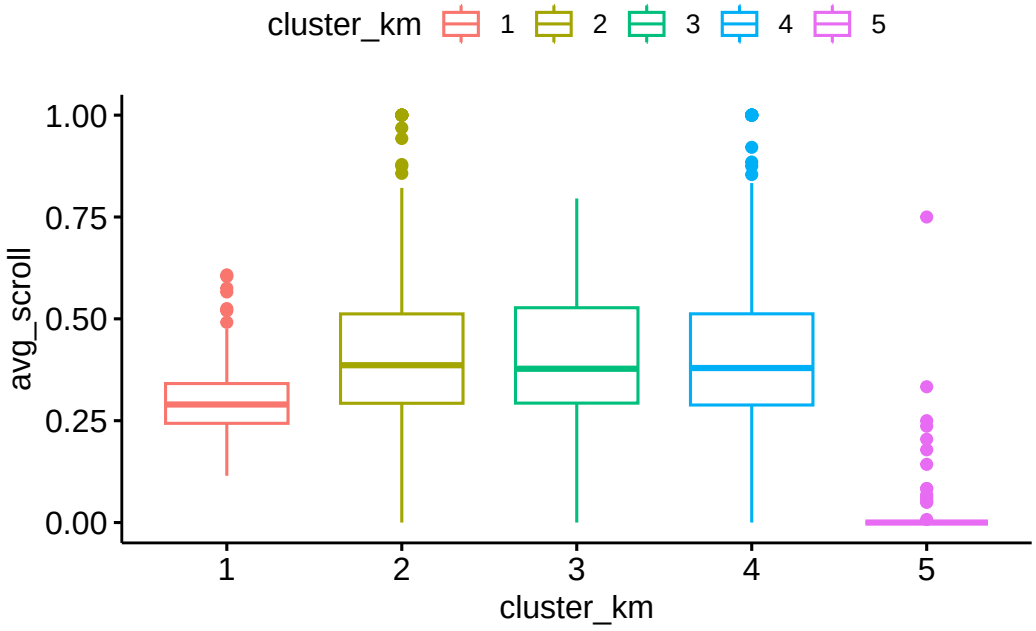
```
# ----------------------------
# 12. Cluster profiling
# ----------------------------
customer_df$cluster_km <- as.factor(customer_df$cluster_km)

cluster_profile <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(across(where(is.numeric), mean, na.rm = TRUE))
```

```
Warning: There was 1 warning in `summarise()`.
i In argument: `across(where(is.numeric), mean, na.rm = TRUE)`.
i In group 1: `cluster_km = 1`.
Caused by warning:
! The `...` argument of `across()` is deprecated as of dplyr 1.1.0.
Supply arguments directly to `.fns` through an anonymous function instead.

  # Previously
  across(a:b, mean, na.rm = TRUE)

  # Now
  across(a:b, \(x) mean(x, na.rm = TRUE))
```

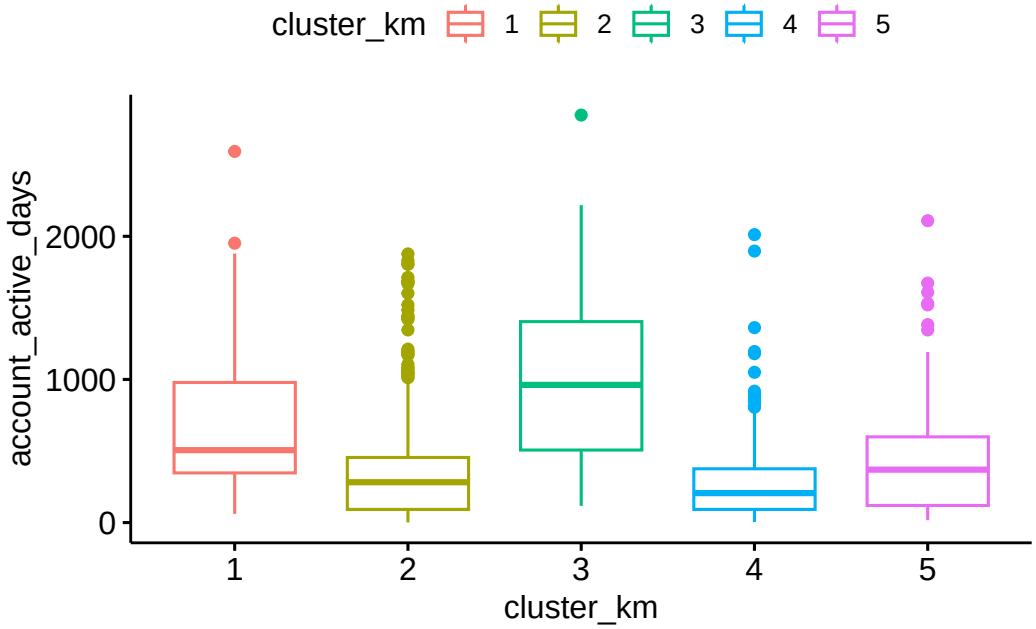
```
# ----------------------------
# 13. Visualisation
# ----------------------------
ggboxplot(customer_df, x = "cluster_km", y = "n_visits",
          color = "cluster_km")
```



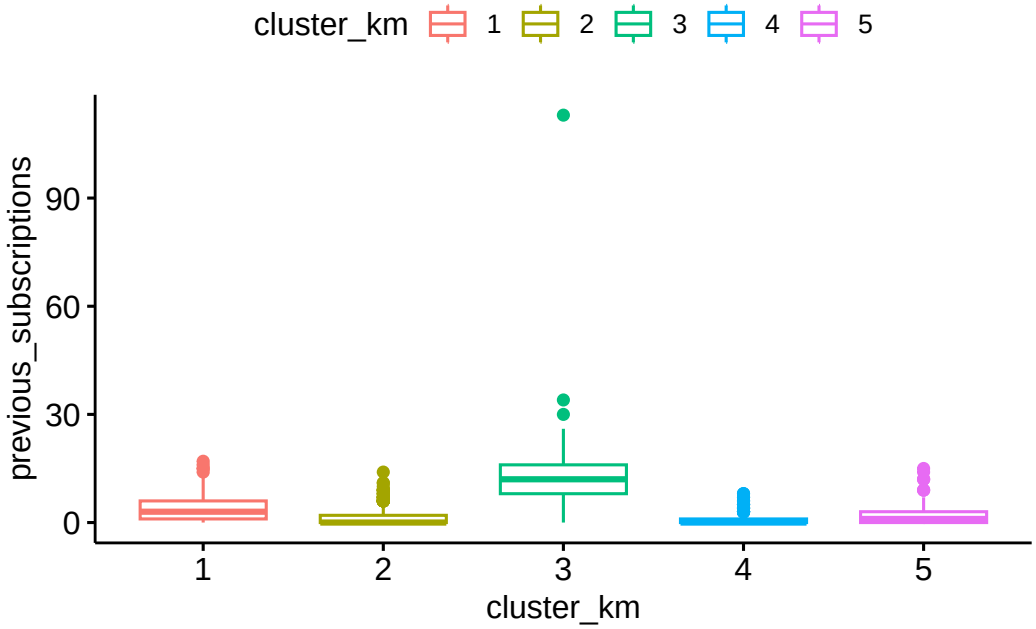

```
ggboxplot(customer_df, x = "cluster_km", y = "avg_scroll",
          color = "cluster_km")
```




```
ggboxplot(customer_df, x = "cluster_km", y = "account_active_days",
          color = "cluster_km")
```




```
ggboxplot(customer_df, x = "cluster_km", y = "previous_subscriptions",
          color = "cluster_km")
```




```
# ----------------------------
# 14. ANOVA tests
# ----------------------------
summary(aov(n_visits ~ cluster_km, data = customer_df))
```

```
              Df    Sum Sq  Mean Sq F value Pr(>F)
cluster_km     4 53943667 13485917   407.3 <2e-16 ***
```

```
Residuals   1270 42052029    33112
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(avg_scroll ~ cluster_km, data = customer_df))
```

```
             Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4  15.52   3.881     145 <2e-16 ***
Residuals   1270  34.00   0.027
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(account_active_days ~ cluster_km, data = customer_df))
```

```
             Df    Sum Sq  Mean Sq F value Pr(>F)
cluster_km    4  64845568 16211392   109.5 <2e-16 ***
Residuals   1270 187992176   148025
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_subscriptions ~ cluster_km, data = customer_df))
```

```
             Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4  15919    3980   208.9 <2e-16 ***
Residuals   1270  24198      19
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_campaigns ~ cluster_km, data = customer_df))
```

```
             Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   2823   705.6     191 <2e-16 ***
Residuals   1270   4693     3.7
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(previous_trials ~ cluster_km, data = customer_df))
```

```
             Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   4996  1249.1    97.3 <2e-16 ***
Residuals   1270  16303    12.8
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
summary(aov(newsletters_before_order ~ cluster_km, data = customer_df))
```

```
             Df Sum Sq Mean Sq F value Pr(>F)
cluster_km    4   1814   453.5   149.5 <2e-16 ***
Residuals   1270   3852     3.0
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
summary(aov(newsletters_after_order ~ cluster_km, data = customer_df))
```

```
              Df Sum Sq Mean Sq F value Pr(>F)
cluster_km     4   1915   478.6   140.9 <2e-16 ***
Residuals   1270   4316     3.4
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```r
# ----------------------------
# 15. CHURN ANALYSIS (IMPORTANT)
# ----------------------------
churn_by_cluster <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(
    churn_rate = mean(churn == 1, na.rm = TRUE),
    n = n()
  )

print(churn_by_cluster)
```

```
# A tibble: 5 x 3
  cluster_km churn_rate     n
  <fct>           <dbl> <int>
1 1               0.470   249
2 2               0.419   430
3 3               0.489   133
4 4               0.369   363
5 5               0.39    100
```

```r
# ----------------------------
# PRINT CLUSTER MEANS (COORDINATES)
# ----------------------------
cluster_summary_long <- customer_df %>%
  group_by(cluster_km) %>%
  summarise(across(where(is.numeric), \(x) mean(x, na.rm = TRUE))) %>%
  pivot_longer(cols = -cluster_km, names_to = "variable", values_to = "mean_value") %>%
  pivot_wider(names_from = cluster_km, names_prefix = "Cluster_", values_from = mean_value)

# This will print the full table in your console
print(cluster_summary_long, n = 50)
```

```
# A tibble: 15 x 6
   variable                Cluster_1 Cluster_2 Cluster_3 Cluster_4 Cluster_5
   <chr>                       <dbl>     <dbl>     <dbl>     <dbl>     <dbl>
 1 account_active_days       666.      373.      998.      273.      455.
 2 previous_subscriptions      4.02      1.53     13.0       0.989     2.27
 3 previous_campaigns          2.33      0.888     5.55      0.565     1.27
 4 previous_trials             1.37      0.574     7.10      0.402     0.91
 5 newsletters_before_order    0.309     0.302     4.23      0.361     0.42
 6 newsletters_after_order     0.345     0.349     4.39      0.435     0.45
 7 n_visits                  613.      128.      215.       77.4       3.7
 8 avg_scroll                  0.300     0.420     0.411     0.426     0.0256
 9 share_mobile                0.679     0.0643    0.425     0.908     0.162
```

```
10 share_desktop               0.279     0.909     0.530     0.0641   0.0927
11 share_restricted            0.277     0.346     0.366     0.393    0.0254
12 share_search                0.0490    0.0678    0.0331    0.0412   0.00126
13 share_internal              0.566     0.540     0.506     0.425    0.0496
14 n_unique_pages            610.      128.      215.       76.8      3.7
15 cluster_hc                  1.02      2.58      2.38      1.92     4.64
```

```
# Definér 5 danske labels baseret på de opdaterede profiler
customer_df <- customer_df %>%
  mutate(cluster_label = case_when(
    cluster_km == 1 ~ "Power-brugeren (Mest mobil)",
    cluster_km == 2 ~ "Desktop-traditionalisten",
    cluster_km == 3 ~ "Veteranen (Høj historik)",
    cluster_km == 4 ~ "Mobil-nykommeren",
    cluster_km == 5 ~ "Spøgelses-brugeren (Lav aktivitet)",
    TRUE ~ "Andet"
  ))

# Tjek den nye fordeling
table(customer_df$cluster_label, customer_df$churn)
```

```
                                    0   1
  Desktop-traditionalisten        250 180
  Mobil-nykommeren                229 134
  Power-brugeren (Mest mobil)     132 117
  Spøgelses-brugeren (Lav aktivitet)  61  39
  Veteranen (Høj historik)         68  65
```

```
# Gem mapping til din Tidymodels-fil
cluster_mapping <- customer_df %>%
  select(pseudo_id, cluster_label)

saveRDS(cluster_mapping, "data/cluster_mapping.rds")

# Tjek fordelingen
table(customer_df$cluster_label)
```

```
          Desktop-traditionalisten                          Mobil-nykommeren
                               430                                       363
      Power-brugeren (Mest mobil) Spøgelses-brugeren (Lav aktivitet)
                               249                                       100
         Veteranen (Høj historik)
                               133
```

3 churn model

```
pacman::p_load(
  tidyverse, tidymodels, themis, ranger, glmnet,
  kernlab, kknn, xgboost, naivebayes, rpart, future, vip
```

```
)

# Indlæs data
model_data <- readRDS("data/model_data.rds")
cluster_mapping <- readRDS("data/cluster_mapping.rds")

# Join cluster labels and clean initial data
model_data_clean <- model_data %>%
  # 1. Join the clusters
  left_join(cluster_mapping, by = "pseudo_id") %>%
  # 2. Convert label to factor for modeling
  mutate(cluster_label = as.factor(cluster_label)) %>%
  # 3. Rens data (fjern leakage og ID'er)
  select(
    -pseudo_id,
    -subscription_cancel_date,
    -type,
    -reason,
    -expiration_date,
    -order_trackertag,
    -first_campaign_day,
    -last_campaign_day,
    -continued_subscription
  ) %>%
  mutate(churn = factor(churn, levels = c("0", "1")))

# Check that cluster_label is present
glimpse(model_data_clean)
```

```
Rows: 1,275
Columns: 14
$ account_active_days      <int> 387, 1330, 61, 92, 581, 1424, 1501, 184, 1256~
$ koen                     <fct> Kvinde, Kvinde, Mand, Kvinde, Kvinde, Mand, M~
$ permission_given_order   <fct> 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
$ permission_given_today   <fct> 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 1, 0, 0, ~
$ previous_subscriptions   <int> 5, 15, 0, 0, 2, 4, 9, 0, 12, 5, 0, 0, 21, 1, ~
$ previous_campaigns       <int> 4, 13, 0, 0, 2, 3, 6, 0, 2, 4, 0, 0, 9, 1, 1,~
$ previous_trials          <int> 2, 1, 0, 0, 0, 0, 4, 0, 6, 1, 0, 0, 10, 0, 0,~
$ newsletters_before_order <int> 1, 0, 0, 0, 0, 0, 5, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ newsletters_after_order  <int> 1, 0, 0, 0, 0, 0, 4, 0, 3, 1, 0, 0, 0, 0, 0, ~
$ churn                    <fct> 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 1, 0, ~
$ early_churn              <dbl> 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, ~
$ age                      <dbl> 76.33699, 69.78630, 39.75342, 45.69041, 74.29~
$ kundetid_gruppe          <fct> 6+ måneder, 6+ måneder, 0-7 dage, 0-7 dage, 6~
$ cluster_label            <fct> Power-brugeren (Mest mobil), Power-brugeren (~
```

```
# Split
set.seed(8)
split <- initial_split(model_data_clean, prop = 0.8, strata = churn)
train_data <- training(split)
test_data  <- testing(split)

# CV
folds <- vfold_cv(train_data, v = 5, strata = churn)
```

```r
# Recipe
rec <- recipe(churn ~ ., data = train_data) %>%
  step_rm(early_churn) %>%   # VERY IMPORTANT: avoid leakage
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_smote(churn)
# step_downsample(churn)

# Models
decision_tree_spec <- decision_tree(
  tree_depth = tune(),
  min_n = tune(),
  cost_complexity = tune()
) %>%
  set_engine("rpart") %>%
  set_mode("classification")

log_reg_spec <- logistic_reg(
  penalty = tune(),
  mixture = tune()
) %>%
  set_engine("glmnet") %>%
  set_mode("classification")

rand_forest_spec <- rand_forest(
  mtry = tune(),
  min_n = tune(),
  trees = 500
) %>%
  set_engine("ranger", importance = "permutation") %>%
  set_mode("classification")

svm_rbf_spec <- svm_rbf(
  cost = tune(),
  rbf_sigma = tune()
) %>%
  set_engine("kernlab") %>%
  set_mode("classification")

xgb_spec <- boost_tree(
  trees = tune(),
  tree_depth = tune(),
  learn_rate = tune(),
  mtry = tune()
) %>%
  set_engine("xgboost") %>%
  set_mode("classification")

# Workflow
workflow_set_obj <- workflow_set(
  preproc = list(rec = rec),
  models = list(
    decision_tree = decision_tree_spec,
```

```
    logistic_reg  = log_reg_spec,
    rand_forest   = rand_forest_spec,
    svm_rbf       = svm_rbf_spec,
    xgboost       = xgb_spec
  )
)

metrics <- metric_set(roc_auc, accuracy, f_meas, sens, spec)


# Train
plan(multisession)

set.seed(8)

# We use suppressMessages to hide the "Fold X: model Y/Z" notes
# and suppressWarnings to hide the Precision/Recall NA warnings.
results <- suppressMessages(suppressWarnings(
  workflow_set_obj %>%
    workflow_map(
      "tune_grid",
      resamples = folds,
      grid = 5,
      metrics = metrics,
      verbose = FALSE, # Switches off the tidymodels progress logger
      control = control_grid(
        save_pred = TRUE,
        save_workflow = TRUE
      )
    )
))

plan(sequential)

#
# plan(multisession)
#
# set.seed(8)
# results <- workflow_set_obj %>%
#   workflow_map(
#     "tune_grid",
#     resamples = folds,
#     grid = 5,
#     metrics = metrics,
#     control = control_grid(
#       verbose = TRUE,
#       save_pred = TRUE,
#       save_workflow = TRUE
#     )
#   )
#
# plan(sequential)
#

# Best model
```

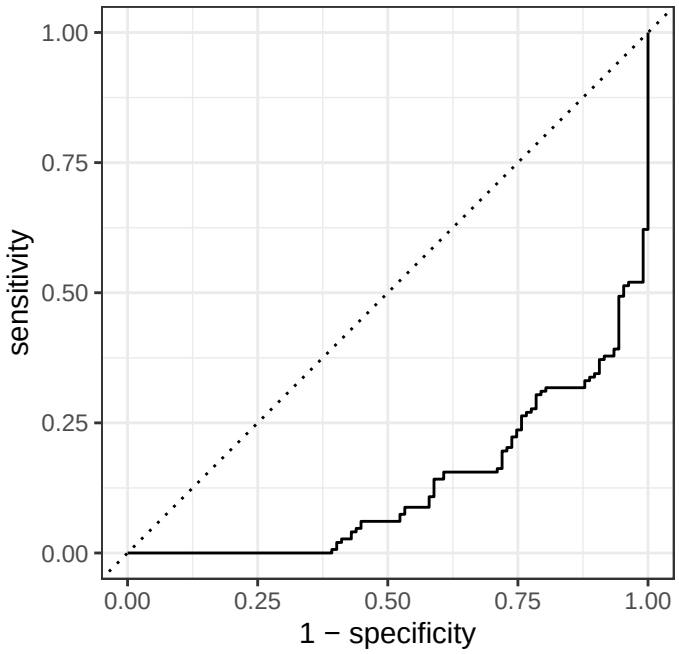
```r
best_model_id <- results %>%
  rank_results(select_best = TRUE) %>%
  filter(.metric == "f_meas") %>%
  slice_max(mean, n = 1) %>%
  pull(wflow_id)

best_tuned <- results %>%
  extract_workflow_set_result(best_model_id) %>%
  select_best(metric = "f_meas")

final_wf <- results %>%
  extract_workflow(best_model_id) %>%
  finalize_workflow(best_tuned)

# Final evaluation
final_fit <- final_wf %>%
  last_fit(split, metrics = metrics)

collect_metrics(final_fit)
```

```
# A tibble: 5 x 4
  .metric  .estimator .estimate .config
  <chr>    <chr>          <dbl> <chr>
1 accuracy binary         0.788 pre0_mod0_post0
2 f_meas   binary         0.821 pre0_mod0_post0
3 sens     binary         0.838 pre0_mod0_post0
4 spec     binary         0.720 pre0_mod0_post0
5 roc_auc  binary         0.869 pre0_mod0_post0
```

```r
test_preds <- collect_predictions(final_fit)

test_preds %>%
  conf_mat(truth = churn, estimate = .pred_class)
```

```
          Truth
Prediction   0   1
         0 124  30
         1  24  77
```

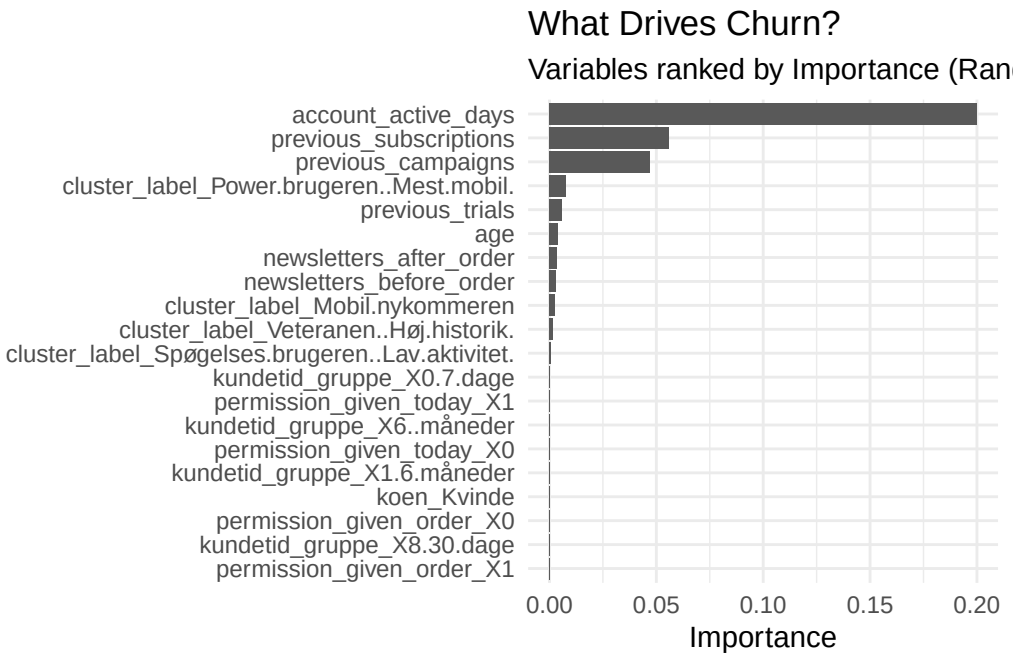
```r
test_preds %>%
  roc_curve(truth = churn, .pred_1) %>%
  autoplot()
```




```
best_model_id
```

```
[1] "rec_rand_forest"
```

```
final_fit %>%
  extract_fit_parsnip() %>%
  vip(num_features = 20) +
  theme_minimal() +
  labs(title = "What Drives Churn?",
       subtitle = "Variables ranked by Importance (Random Forest)")
```



```r
################################################
#   MODEL 2: EARLY CHURN (TASK 3)
################################################

# Only customers who continued
early_data <- model_data_clean %>%
  filter(churn == "0") %>%
  mutate(early_churn = factor(early_churn, levels = c("0", "1")))

# Split
set.seed(8)
split_early <- initial_split(early_data, prop = 0.8, strata = early_churn)
train_early <- training(split_early)
test_early  <- testing(split_early)

# CV
folds_early <- vfold_cv(train_early, v = 5, strata = early_churn)

# Recipe
rec_early <- recipe(early_churn ~ ., data = train_early) %>%
  step_rm(churn) %>%  # remove parent target
  step_novel(all_nominal_predictors()) %>%
  step_dummy(all_nominal_predictors(), one_hot = TRUE) %>%
  step_zv(all_predictors()) %>%
  step_normalize(all_numeric_predictors())
#  step_downsample(early_churn)

# Workflow (reuse same models!)
workflow_set_early <- workflow_set(
  preproc = list(rec = rec_early),
  models = list(
    decision_tree = decision_tree_spec,
    logistic_reg  = log_reg_spec,
    rand_forest   = rand_forest_spec,
    svm_rbf       = svm_rbf_spec,
    xgboost       = xgb_spec
  )
)

# Train
plan(multisession)

set.seed(8)
results_early <- workflow_set_early %>%
  workflow_map(
    "tune_grid",
    resamples = folds_early,
    grid = 5,
    metrics = metrics
  )
```

```
Warning: package 'future' was built under R version 4.5.3
```

```
Warning: package 'tune' was built under R version 4.5.2
```

```
Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

Warning: package 'future' was built under R version 4.5.3

Warning: package 'tune' was built under R version 4.5.2

i Creating pre-processing data to finalize 1 unknown parameter: "mtry"

→ A | warning: ! 23 columns were requested but there were 22 predictors in the data.
                 22 predictors will be used.

maximum number of iterations reached 0.0001029682 0.0001028812maximum number of iterations reached 0.00

i Creating pre-processing data to finalize 1 unknown parameter: "mtry"
```

```r
plan(sequential)

# Best model
best_model_id_early <- results_early %>%
  rank_results(select_best = TRUE) %>%
  filter(.metric == "f_meas") %>%
  slice_max(mean, n = 1) %>%
  pull(wflow_id)

best_tuned_early <- results_early %>%
  extract_workflow_set_result(best_model_id_early) %>%
  select_best(metric = "f_meas")

final_wf_early <- results_early %>%
  extract_workflow(best_model_id_early) %>%
  finalize_workflow(best_tuned_early)

# Final evaluation
final_fit_early <- final_wf_early %>%
  last_fit(split_early, metrics = metrics)

collect_metrics(final_fit_early)
```

```
# A tibble: 5 x 4
  .metric  .estimator .estimate .config
  <chr>    <chr>          <dbl> <chr>
1 accuracy binary         0.799 pre0_mod0_post0
2 f_meas   binary         0.871 pre0_mod0_post0
3 sens     binary         1     pre0_mod0_post0
4 spec     binary         0.375 pre0_mod0_post0
5 roc_auc  binary         0.899 pre0_mod0_post0
```

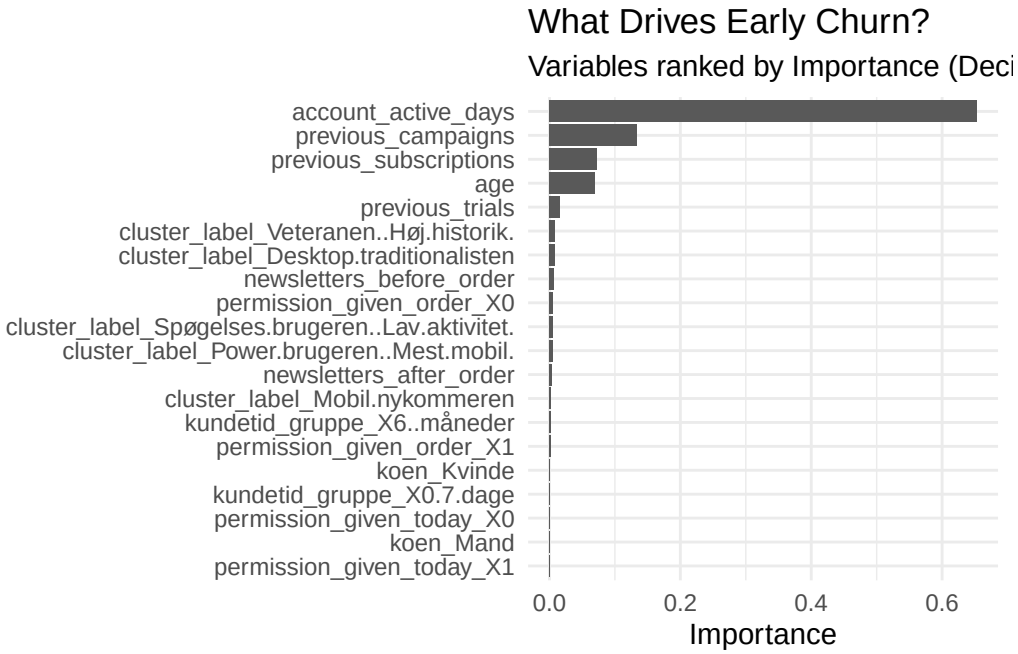
```
test_preds_early <- collect_predictions(final_fit_early)

test_preds_early %>%
  conf_mat(truth = early_churn, estimate = .pred_class)
```

```
          Truth
Prediction   0   1
         0 101  30
         1   0  18
```

```
best_model_id_early
```

```
[1] "rec_xgboost"
```

```
final_fit_early %>%
  extract_fit_parsnip() %>%
  vip(num_features = 20) +
  theme_minimal() +
  labs(title = "What Drives Early Churn?",
       subtitle = "Variables ranked by Importance (Decision Tree)")
```



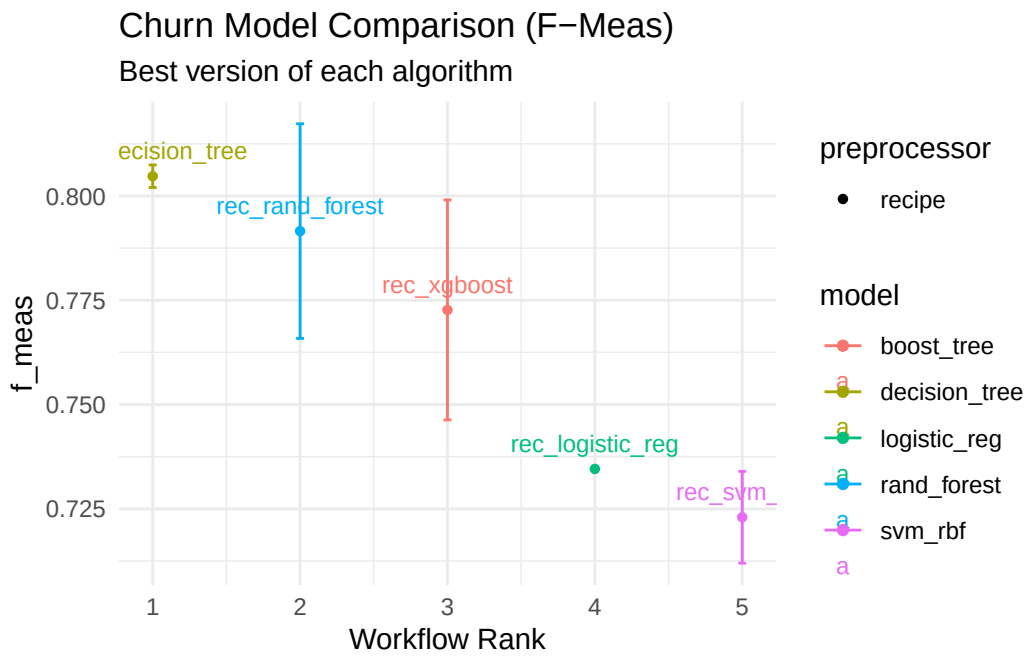

```
####################################################
#   RESULTS VISUALIZATION & COMPARISON
####################################################

# 1. Show the "Leaderboard" for Model 1 (Churn)
# This lists all models ranked by F-Measure
cat("\n--- Model 1: Churn Leaderboard ---\n")
```

```
--- Model 1: Churn Leaderboard ---
```

```r
results %>%
  rank_results(rank_metric = "f_meas", select_best = TRUE) %>%
  select(wflow_id, .metric, mean, std_err, rank) %>%
  filter(.metric == "f_meas") %>%
  print()
```

```
# A tibble: 5 x 5
  wflow_id           .metric  mean std_err  rank
  <chr>              <chr>   <dbl>   <dbl> <int>
1 rec_decision_tree  f_meas  0.805 0.00164     1
2 rec_rand_forest    f_meas  0.792 0.0156      2
3 rec_xgboost        f_meas  0.773 0.0160      3
4 rec_logistic_reg   f_meas  0.735 0           4
5 rec_svm_rbf        f_meas  0.723 0.00668     5
```

```r
# 2. Plot Model 1 Comparison
# This gives you a visual look at how the different engines compared
p1 <- autoplot(results, metric = "f_meas", select_best = TRUE) +
  geom_text(aes(label = wflow_id), vjust = -1, size = 3) +
  labs(title = "Churn Model Comparison (F-Meas)",
       subtitle = "Best version of each algorithm") +
  theme_minimal()

print(p1)
```



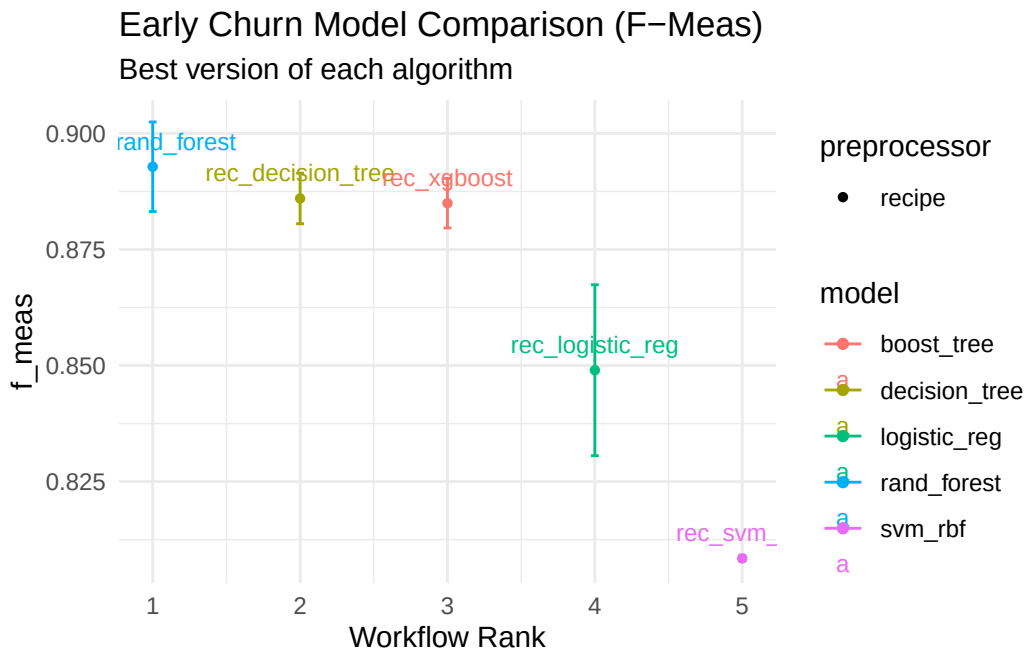

```r
# 3. Show the "Leaderboard" for Model 2 (Early Churn)
cat("\n--- Model 2: Early Churn Leaderboard ---\n")
```

```
--- Model 2: Early Churn Leaderboard ---
```

```
results_early %>%
  rank_results(rank_metric = "f_meas", select_best = TRUE) %>%
  select(wflow_id, .metric, mean, std_err, rank) %>%
  filter(.metric == "f_meas") %>%
  print()
```

```
# A tibble: 5 x 5
  wflow_id          .metric  mean  std_err  rank
  <chr>             <chr>   <dbl>    <dbl> <int>
1 rec_rand_forest   f_meas  0.893 0.00586      1
2 rec_decision_tree f_meas  0.886 0.00331      2
3 rec_xgboost       f_meas  0.885 0.00324      3
4 rec_logistic_reg  f_meas  0.849 0.0112       4
5 rec_svm_rbf       f_meas  0.808 0.000384     5
```

```
# 4. Plot Model 2 Comparison
p2 <- autoplot(results_early, metric = "f_meas", select_best = TRUE) +
  geom_text(aes(label = wflow_id), vjust = -1, size = 3) +
  labs(title = "Early Churn Model Comparison (F-Meas)",
       subtitle = "Best version of each algorithm") +
  theme_minimal()

print(p2)
```




```
# 5. Identify exactly which Hyperparameters the winner used
cat("\n--- Best Model Hyperparameters ---\n")
```

```

--- Best Model Hyperparameters ---
```

```r
cat("Churn Model:", best_model_id, "\n")
```

```
Churn Model: rec_rand_forest
```

```r
print(best_tuned)
```

```
# A tibble: 1 x 3
   mtry min_n .config
  <int> <int> <chr>
1    23    21 pre0_mod5_post0
```

```r
cat("\nEarly Churn Model:", best_model_id_early, "\n")
```

```
Early Churn Model: rec_xgboost
```

```r
print(best_tuned_early)
```

```
# A tibble: 1 x 5
   mtry trees tree_depth learn_rate .config
  <int> <int>      <int>      <dbl> <chr>
1    17   500         11      0.001 pre0_mod4_post0
```

```r
#
# # 1. Get predictions for all users
# final_results <- final_fit %>%
#   extract_workflow() %>%
#   augment(model_data_clean)
#
# # 2. Create the Cluster-Risk Matrix
# risk_profile <- final_results %>%
#   group_by(cluster_label) %>%
#   summarise(
#     n_users = n(),
#     # Model's average predicted probability of churn
#     avg_predicted_risk = mean(.pred_1),
#     # Actual churn recorded in data
#     actual_churn_rate = mean(churn == "1"),
#     # Early Churn rate within this cluster
#     early_churn_rate = mean(early_churn == 1),
#     # Loyalty score (How many stayed)
#     loyalty_rate = mean(churn == "0")
#   ) %>%
#   arrange(desc(avg_predicted_risk))
#
# print(risk_profile)
```